# LaTeX Indexing Editor

*Design Overview — subsystems and classes (draft)*

## 1. Introduction

The LaTeX Indexing Editor is a PySide6 desktop application for building and maintaining embedded indexes in LaTeX source. It opens a folder of .tex files as a project, scans them for \index macros, and presents everything it finds in two coordinated views — a hierarchical index tree and a spreadsheet-style entry table — through which entries can be created, edited, re-filed and deleted. Every change is written back into the LaTeX source itself; the application adds a working layer over the document rather than replacing it with a database of its own.

Each project carries a SQLite database alongside its .tex files, holding the parsed index data and the project's own settings. That database is a cache and a coordinate map, not the source of truth: the .tex files are authoritative, and the application can rebuild the database from them at any time.

### Design principles

- Layer separation. The code is organised as models, views and controllers, and the boundary is enforced by convention throughout: views hold no business logic and never touch persistence, models import no Qt widgets, and controllers own all coordination.
- Signals over direct calls. Subsystems communicate through Qt signals rather than holding references to each other, so a view can be replaced or driven from a test without the controller knowing the difference.
- Long work goes off the UI thread. Project loading, project-wide search and the LaTeX export pipeline each run in a worker object inside a dedicated QThread, following one shared worker/thread pattern. The authority-record lookup behind name inversion is the one exception: it is short-lived and a single call deep, so it runs on a small thread pool and returns through a queued signal rather than carrying a thread of its own.
- Deferred, transactional writes. Index changes accumulate in memory and are written to the database in a single transaction when the project is saved, so a save is all-or-nothing.
- Undo is a first-class model. Every operation that mutates the index records a command describing both halves of the change, so it can be reversed across the source file, the database and the views together.

### Architectural shape

One controller sits at the centre. AppPipelineController is constructed once at startup, is handed every model and view, and wires the signal graph that connects them. Feature-specific controllers hang off it, each owning one tool or one pane. Subsystems below are grouped by responsibility rather than by

folder; models, views and controllers belonging to the same feature are described together.

Data flows in a consistent direction. A user action reaches a view, which emits a signal; a controller interprets it, asks a model to change state, and writes through to the .tex source via the document I/O layer; the model then signals back and the views refresh. Persistence is touched only at the ends of that chain — on project load, and on save.

# 2. Subsystems

## 2.1 Application Shell and Orchestration

The frame the rest of the application is mounted into, and the object that assembles it. The shell classes are deliberately passive: the main window owns layout and nothing else, exposing its panels for the pipeline controller to populate. This is what allows the entire object graph to be constructed headlessly in tests exactly as main.py constructs it.

**`AppPipelineController`** — The central orchestrator. Constructs and connects every controller, model and view, owns the project open/save/close workflows, the recently opened project list, the undo stack, the auto-save timer, the off-thread name lookups, and the exit prompts.

**`LatexEditor`** — The main window. A passive layout frame that exposes its menu bar, toolbar, status bar, sidebar and tab area for the controller to populate.

**`MainMenuBar`** — The menu bar. Emits a named request signal per action, enables or disables project-scoped items on demand, and rebuilds the Open Recent submenu each time that submenu is about to be shown.

**`MainToolBar`** — The toolbar. Hosts the font, size and dark-mode controls and broadcasts their changes.

**`MainStatusBar`** — The status bar. Paints pre-formatted messages; holds no logic of its own.

**`ProjectSidebarView`** — The left sidebar. A tab container isolating the file tree and index tree from the main window's layout.

**`AppStyleConfiguration`** — Central style authority. Generates stylesheets and palettes from the active theme colours for every widget that needs one.

**`ThemeChangedSignals`** — The application-wide event channel that broadcasts a theme change to any widget listening for one.

**`SessionLogger`** — Captures stdout and stderr across all layers, timestamps them, and writes a per-session log file that follows the open project.

***Relates to:*** *everything. This subsystem holds references to all others; they do not hold references back to it, communicating upward by signal instead.*

## 2.2 Project and Workspace Management

Owns what a project is: which folder, which .tex files are in scope, and where the project database lives. Once a project has been opened, the database is trusted as the record of the workspace rather than the folder being re-walked on every open, which is what keeps reopening a large project fast. The cost of that choice is that changes made outside the application need an explicit resync, and this subsystem provides both the detection and the escape hatch.

**`ProjectScopeController`** — Owns the active project: its name, root path, database handle, and which files are in or out of indexing scope.

**`FileTreePersistence`** — The SQLite layer. Owns the schema and every read and write for references, headings, cross-references, file records, sync checksums and project metadata; also provides the transaction used by the save drain.

**`_TransactionConnection`** — Internal proxy standing in for a database connection while a transaction is open, so the many methods that would otherwise commit individually join one transaction instead.

**`ProjectLoadWorker`** — Runs a project open off the UI thread: walks or reads the file list, parses every .tex file for index macros, and returns a complete dataset.

**`SafeProjectLoadThread`** — Thread container for the loader, isolating its database connections and forwarding its signals safely.

**`FileTreeView`** — The Workspace Files tree. Presents the project folder and emits open, set-base-file and prune requests.

**`LatexFolderFilterProxy`** — Filter model that hides non-LaTeX files from the tree without collapsing the folder hierarchy around them.

**`PrunedFilesController`** — Owns the Manage Pruned Files tool: lists files excluded from scope and restores the ones the user selects.

**`PrunedFilesDialog`** — The checklist view for that tool. Emits a restore request; never touches the database.

**`ExternalFileWatcherEngine`** — Watches registered project files and reports changes made outside the application.

**`SessionBackupManager`** — Keeps a pristine copy of each file before the application first writes to it, so Discard can restore it. Cleared on save.

***Relates to:*** *the Index Data subsystem, which it loads and persists; Document I/O, which registers files with the backup manager and the watcher before writing to them.*

## 2.3 Document I/O and Editor Tabs

The only code that writes to a .tex file. Every index operation ultimately becomes a macro span rewrite here, and this subsystem decides where that write lands: into a tab's live buffer when the file is open, or directly to disk when it is not. It also tracks whether each file's recorded coordinates still match its contents, which is what lets a save know which checksums it may safely re-stamp.

**`DocumentIOController`** — Owns all file reads and writes: macro span rewrites, insertions, buffer commits, per-tab discard, and coordinate-sync tracking.

**`WorkspaceLifecycleController`** — Owns tab lifecycle: opening files into tabs, the per-tab unsaved-changes prompts, and closing tabs on project close.

**`IndexNavigationHelper`** — Shared navigation service that focuses a file and jumps to a coordinate,

so any view can request navigation without coupling to the tab machinery.

**`EditorTab`** — A .tex file's view. Looks like an editor but restricts input to navigation, copy, find and undo/redo; all mutation arrives programmatically through the pipeline.

**`LatexHighlighter`** — Syntax highlighting for the tab's LaTeX content.

**`TextSanitizer`** — Path and text normalisation, including the newline normalisation that the character-offset coordinate convention depends on.

**`TabFindDialog`** — The floating, theme-aware find bar for the active tab.

**`CustomVectorButton`** — The find bar's flat vector-drawn button.

***Relates to:*** *Index Editing, which calls into it for every macro rewrite; Project Management, whose backup manager and file watcher it notifies before each write.*

## 2.4 Index Data Model and Persistence

The in-memory representation of the index, and the rules for parsing and rebuilding it. These classes are Qt-free apart from signals and know nothing about widgets. Two of them carry most of the subsystem's weight: the reference cache, which holds every \index occurrence with its file coordinates, and the tree engine, which owns heading identity. Both journal their pending database writes rather than performing them, leaving the actual writing to a single transaction at save time.

**`EntryModifierModel`** — The live cache of every index reference: heading text, page style, file coordinates and range partners. Journals its own pending inserts, updates and deletes.

**`IndexTreeModelEngine`** — Owns the heading hierarchy and is the single allocator of heading identifiers, so a heading can exist and be referenced before its row is written.

**`PendingChangesJournal`** — Entity-keyed record of unwritten database operations for one table, collapsing repeated edits and cancelling out changes that undo each other.

**`IndexEditStagingModel`** — Session-only staging for an edit still in progress in the entry table, tracking each entry's baseline against its current value.

**`_StagedEntry`** — Internal per-entry record of that baseline and staged value.

**`LatexIndexParser`** — Scans raw LaTeX text for index macros, skipping comments, optional arguments and macro definitions.

**`IndexTag`** — A parsed \index tag: its heading levels, sort keys and encap, with serialisation back to macro text.

**`XRefSpec`** — A parsed see / see also encap.

**`Finding`** — One thing worth saying about a span of entry text: what LaTeX or makeindex will read as grammar rather than as text, where it sits, and what to write instead. Purely advisory, and deliberately so — a bare per-cent sign compiles clean with no warning anywhere and silently truncates the printed entry, so the checker producing these exists to say so, not to refuse the entry.

**`IndexEntryModel`** — A single entry being composed in the Index Entry window, before it becomes a

macro. Each level carries its display text and, separately, the sort key the indexer chose for it; no sort key is ever inferred from formatting.

**`ReferenceCarrier`** — A mutable box used to return a value through a Qt signal.

**`MacroIDGenerator`** — Allocates the unique identifier carried by each reference.

***Relates to:*** *Project Management, which loads and saves it; Index Editing, which is the only subsystem permitted to mutate it; Index Presentation, which renders it.*

## 2.5 Index Editing and Undo

Where an index change actually happens. Every path — the Index Entry window, an inline tree rename, a table cell edit, a bulk delete, a tool-applied fix — converges on this subsystem, which updates the source file, the in-memory model and the views as one operation and records a command that can reverse all three. Keeping that convergence is deliberate: it is why undo can be trusted, and why an edit cannot leave the .tex file and the database disagreeing.

**`IndexEditController`** — The rewrite pipeline. Owns heading renames, cell edits, single and bulk deletions, coordinate shifting after each write, and the discard path for unsaved work.

**`LatexIndexController`** — Owns the Index Entry window: assembles a new macro from the entered fields and inserts it at the cursor or around the selection.

**`IndexTreeController`** — Routes index tree interaction to the edit pipeline; holds no parsing or presentation logic.

**`EntryModifierController`** — Routes entry table interaction, mediating the staging model so a part-typed cell is not committed early.

**`IndexCommandStack`** — The undo and redo stacks. Bounded to a user-configurable depth, and executed in peek-perform-complete steps so a failed write leaves the stacks intact.

**`IndexCommand`** — One undoable operation, with a user-facing label and everything needed to invert it.

**`MacroEdit`** — One macro span write within a command: position, text before, text after.

**`EntrySnapshot`** — A complete copy of a reference record and its tree position, enough to recreate it from nothing.

**`HeadingChange`** — One entry's heading text before and after a rename.

***Relates to:*** *Document I/O for the file write; Index Data for the model update; Index Presentation for the view refresh. The Tools subsystem applies its fixes by calling in here rather than editing directly.*

## 2.6 Index Presentation

The two views of the index and the widgets that support them. Both are pure presentation: they render what the model gives them, emit signals when the user acts, and are refreshed by controllers. The delegates exist because index text is LaTeX — it has to be displayed as it will print, while remaining editable as source.

**`IndexTreeView`** — The hierarchical index view, sorted case-insensitively, with inline renaming and navigation to source.

**`CaseInsensitiveItem`** — Tree item providing that sort order and placing cross-reference nodes ahead of ordinary entries.

**`IndexTextFormatterDelegate`** — Renders LaTeX bold and italic within a tree label while preserving hierarchy indentation. It also italicises the leading See or See also label of a cross-reference node, which is the only styling the application itself imposes on a label; the target after it keeps whatever formatting its own entry asks for.

**`IndexLinkDelegate`** — Renders a reference's page numbers as clickable links to their source positions.

**`EntryModifierList`** — The entry table: one row per reference, with display text, sort keys and page style as separate sortable columns.

**`EntryModifierTableView`** — Table subclass supplying the row-finalisation signal Qt does not provide for every edit-completion case.

**`PageStyleDelegate`** — Presents the page style column as a Standard / Bold / Italic choice, and locks the structural range markers against editing.

**`LatexIndexWindow`** — The docked Index Entry window where a new entry is composed, one field per heading level plus the sort field beside it, which is offered as soon as a level carries formatting and on demand for every level. Both this window and the entry table mark a field or cell whose text carries an index-grammar or LaTeX special character, through one shared presentation module, so that creating an entry and editing one cannot say different things about the same text.

**`EntryWindowTitleBar`** — Its custom title bar, replacing the native dock header.

**`CustomLineEdit`** — Its entry field, detecting backspace in an empty sub-entry to step back a level.

**`SortKeyLineEdit`** — The sort field beside a heading level. Mirrors that level's display text with its formatting read out until the indexer types in it, after which nothing rewrites it — including clearing it, an empty field being a decision to file under the display text exactly as written.

**`LatexEntryAutoCompleter`** — Suggests existing headings while typing a new entry.

**`TabCompletionEventFilter`** — Makes Tab accept the highlighted suggestion.

**`BaseContextMenuManager`** — Shared context-menu construction and theming.

**`IndexTreeContextMenuManager`** — The index tree's menu: delete term, and the tree-side actions.

**`FileTreeContextMenuManager`** — The file tree's menu: open, set as base file, prune.

**`EditEntryContextMenuManager`** — The entry table's menu: invert name, delete, duplicate, invert headings, resolved against the current selection.

***Relates to:*** *Index Data for what it displays; Index Editing, to which it forwards every user action; Configuration, which supplies its theme and the page styles it recognises.*

## 2.7 Cross-References

See and see-also pointers are managed apart from ordinary entries because they have no page reference and no natural home in the source. They live in their own database table and are emitted into a generated cross_refs.tex, which is excluded from index scanning and linked into the base document once. Because that generated file cannot be rebuilt by re-scanning, cross-reference edits are written immediately rather than deferred to save — the one deliberate exception to this application's deferred-write rule.

**`CrossReferenceController`** — Owns the Cross-References tab, the generated file, its injection into the base document, and migration of legacy inline pointers.

**`CrossReferenceList`** — The tab's view: an add row and an editable table of every cross-reference in the project.

**`XrefTypeDelegate`** — Presents the see / see also choice as a dropdown.

**`LegacyXrefMigrationDialog`** — Reviewable checklist of old-style inline cross-references found in the source, for moving into the managed system.

***Relates to:*** *Index Presentation, where managed cross-references appear as read-only tree nodes; Project Management for the base file the generated file is linked into.*

## 2.8 Tools and Analysis

Self-contained features reached from the Tools menu, each following the same shape: a controller that analyses the project and a view that presents reviewable results for the user to accept or decline. None of them writes to the source directly — fixes are applied through the index editing pipeline, so a tool-applied change is undoable and coordinate-safe like any other.

**`RangeConsistencyController`** — Scans for range-pairing faults — orphaned openers or closers, overlapping ranges, enclosed point references — and applies the fixes selected.

**`RangeConsistencyDialog`** — The grouped, checkable result list for that scan.

**`IndexStatisticsDialog`** — Read-only summary of the index: heading counts by level, total references, cross-references.

**`HeadNoteDialog`** — Collects the head-note text spliced into the base document ahead of the printed index.

**`NameInverter`** — Converts a personal name to indexed order, combining a local rule-based conversion with an authority-record lookup, and remembering corrections. It is synchronous; the pipeline controller is what runs it off the UI thread. Life dates are stripped from an authority heading before it is offered, an index not filing under them.

**`NameInversionResult`** — The outcome of one inversion: the authority heading, the rule-based suggestion, and which of the two was used.

**`NameInversionDialog`** — Presents both suggestions for the user to choose from or overrule.

**`SearchWorker`** — Project-wide search across the in-scope file list, off the UI thread.

**`SafeSearchThread`** — Thread container for that worker.

**`AdvancedSearchWindow`** — The search window hosting both search modes and emitting navigation requests for results.

**`ExactSearchPanel`** — Literal phrase matching options.

**`FuzzySearchPanel`** — Approximate matching options, with a similarity threshold.

***Relates to:*** *Index Data, which every tool reads; Index Editing, through which every fix is applied; Document I/O for navigation to a result.*

## 2.9 LaTeX Output and Export

Turns the project into something readable outside the application. The export runs the real LaTeX toolchain rather than formatting the database directly — a compile pass to resolve every \index call, the configured index engine to sort the result, then a conversion to RTF — so what it produces is what the indexing engine would actually print. The same configuration subsystem generates the preamble the document needs for that to work.

**`IndexExportController`** — Coordinates the compile, index, parse and render pipeline for one export.

**`RtfExportEngine`** — Runs the external toolchain steps and reports progress through them.

**`RtfExportMetadata`** — The configuration and executable paths for a single export run.

**`RtfExportWorker`** — Runs the pipeline off the UI thread, since the external tools block.

**`RtfExportThread`** — Thread container for that worker.

**`RtfExportView`** — Writes the sorted index out as Rich Text, carrying bold and italic page styles and non-ASCII characters through.

**`RtfViewerDialog`** — Read-only preview of the generated file.

***Relates to:*** *Configuration, which supplies the compiler and engine paths and generates the preamble; Project Management for the base document and output folder.*

## 2.10 Configuration and Preferences

Two scopes of setting, deliberately kept apart. Application-scoped settings — fonts, the undo depth, the auto-save interval, the log folder — are global and stored in the platform's settings store. Project-scoped settings — the LaTeX and indexing options, and theme colours — are copied into each project's own database the first time it is opened, so changing them for one book leaves every other project untouched. The list of recently opened projects belongs to the application scope as well, being a record of use rather than a project setting.

**`PreferencesPersistence`** — The application-scoped settings store, with the coercion and migration needed to read values back reliably across platforms. It also keeps the recently opened project list, stored deeper than the number of entries the File menu shows, so lowering that number hides the

oldest rather than discarding them.

**`IndexPrefsConfigController`** — Owns the Preferences dialog and routes each tab's payload to the correct scope.

**`IndexPrefsConfigDialog`** — The dialog: General, LaTeX Settings, UI Themes and RTF Export.

**`IndexPrefsConfigModel`** — The working copy of the LaTeX and indexing options, and the generator that turns them into preamble text.

**`IndexPrefsData`** — The typed record of those options with their defaults.

**`ThemeConfigModel`** — The mutable runtime state of both colour sets; Qt-free.

**`DarkThemeColours / LightThemeColours`** — The two colour records and their factory defaults.

**`ThemeConfigController`** — Mediates theme state between the model, both persistence scopes and the style authority.

**`ThemeConfigDialog`** — Standalone theme editor, sharing its tab widgets with the Preferences dialog.

**`_ColourRow / _ThemeTab / _PreviewPanel`** — One colour swatch row, one theme's editor, and the live preview beside it.

**`LatexCommandRegistryModel`** — The global library of user-defined LaTeX commands, and the filter deciding which qualify as indexing commands.

**`CreateCommandController`** — Owns creating a command, by hand or through the wizard.

**`CreateCommandDialog`** — The two-field name and definition editor.

**`LatexCommandWizardDialog`** — The guided alternative for users who would rather not write the declaration.

**`ProjectCommandManagerController`** — Owns which of the global commands the current project has adopted.

**`ProjectCommandManagerDialog`** — The two-list adopt and remove view.

***Relates to:*** *the Application Shell, which applies the settings live; Export, which needs the toolchain paths; Index Presentation, which takes its theme and its recognised page styles from here.*

## 2.11 Help

In-application documentation, bundled with the executable and read from the install location rather than from any project. Topics are authored as Markdown and rendered on demand, which keeps the shipped content editable as plain text.

**`HelpController`** — Owns the help window and the About box, and resolves the bundled content relative to the application's own location.

**`HelpViewerWindow`** — Contents tree, topic pane and history navigation.

**`MarkdownTextBrowser`** — Renders a Markdown topic to HTML on demand and resolves cross-topic links against the help root.

**`AboutDialog`** — The About box, reporting the version being run along with the Python and Qt versions worth quoting in a bug report.

***Relates to:*** *the Application Shell only. Help is intentionally isolated — it has no project scope and no dependency on any other subsystem.*

## 3. How the subsystems fit together

Four flows account for most of the interaction between subsystems. Following them is the quickest way to understand where any given piece of behaviour lives.

**Opening a project**

Project Management resolves the folder and database, then runs the loader on a worker thread. The parsed result is ingested by the Index Data subsystem and handed to Index Presentation to display. Configuration seeds the project's settings from the global defaults on first open and loads them from the project's database thereafter. The Application Shell enables project-scoped menu items, starts the auto-save timer, and records the project in the recently opened list once the load has actually succeeded. A remembered project is checked for existence only when it is chosen, rather than every time the File menu opens.

**Editing an entry**

Index Presentation emits the user's action. Index Editing interprets it, asks Document I/O to rewrite the macro span in the source, updates the Index Data model and shifts the coordinates of everything after it in that file, records an undo command, and refreshes the views. The database write is not performed — it is journalled.

**Saving**

The Application Shell commits every open tab's buffer through Document I/O, then drains both journals into the database in a single transaction via Project Management. It re-stamps each written file's checksum so the application's own work is not later reported as an outside change, and clears the session backups, which moves the point a Discard reverts to. Auto-save runs this same workflow on a timer.

**Exporting**

Configuration supplies the toolchain paths and the generated preamble; Project Management supplies the base document and output folder; Export runs the external tools off the UI thread and renders the sorted result. Because it compiles the real files on disk, it reflects the last save rather than unsaved work.

### Dependency direction

The layering holds in one direction. Views depend on nothing but Qt and the style authority. Models depend on nothing but each other and the persistence layer. Controllers depend on both and are the only classes that know how a feature is assembled. Where two subsystems must cooperate, they do so through a controller and a signal rather than a direct reference — which is why the tools, the export pipeline and the editing pipeline can each be exercised in isolation.

## Deliberate exceptions

- Cross-references write to the database immediately, because their generated .tex file cannot be rebuilt by re-scanning the source.
- Project configuration writes immediately, because a setting such as the base file would be wrong not to take effect until save.
- A file with no open tab is rewritten on disk as it is edited, while a file open in a tab is changed in its buffer and reaches disk on save.

# 4. The databases

The application keeps two SQLite databases; one is created per project and lives beside the .tex files it describes, the other is application-wide and lives with the installed program. Neither holds anything the application could not do without. The project database is a cache and a coordinate map over source that remains authoritative; the names database is a lookup cache. Both can be deleted, and the cost is rebuild time rather than lost work.

## 4.1 The project database

One file per project, written into the project folder as <project name>_index_manifest.db and found again on reopen by that suffix. It is created the first time a folder is opened as a project and thereafter carries four kinds of state: which files are in scope, the parsed index and where every macro sits in its file, the managed cross-references, and the project's own settings.

Its purpose is to make reopening a project cheap and to give each project its own configuration. On the first open the folder is walked and every .tex file parsed; on later opens the file list and the index are read back from here instead, and the folder is only re-walked on an explicit resync. Because the .tex files remain the source of truth, nothing in this database is irreplaceable — a rescan rebuilds it. The cost of that trade is that edits made outside the application go unnoticed until detected, which is what the recorded per-file checksums are for.

Most of it is written at save time. Index changes accumulate in memory as journalled operations and are drained into project_headings and project_references in a single transaction, so a save is all-or-nothing. Two things deliberately break that rule and write immediately:

- project_cross_references, because the cross_refs.tex file generated from it cannot be rebuilt by re-scanning the source.
- project_metadata, because a setting such as the base file would be wrong not to take effect until the next save.

The schema is seven tables.

### `project_metadata`

The project's own settings, as a key/value store rather than a fixed set of columns, so a new preference needs no schema change. It holds the project name, the base .tex file, the compiler and index-engine paths, the output folder, the head-note text, the LaTeX and indexing options and theme colours copied from the global defaults on first open, and flags recording one-time decisions such as a declined cross-reference migration.

| Field | Purpose |
|---|---|
| **key** | The setting's name. Primary key, so a setting appears exactly once. |
| **value** | The setting's value, always stored as text; the reading side coerces it back to the type it expects and tolerates values written by an older version. |
| **updated_at** | When that key was last written. |

### project_files

Every file the loader has seen in the project folder, and whether it counts as being in indexing scope. This table is what allows the Workspace Files tree to be rebuilt from the database rather than by walking the folder again on every open.

| Field | Purpose |
|---|---|
| **absolute_path** | Full path to the file. Primary key, and the identity every other part of the application refers to the file by. |
| **file_name** | The bare file name, used for display in the tree. |
| **is_active** | 1 if the file is in indexing scope, 0 if it has been pruned. Manage Pruned Files toggles this rather than deleting the row, so a pruned file can be restored. |
| **last_indexed** | When the row was last written. |

### project_headings

One row per distinct heading path in the index. Separating headings from the occurrences that use them is what gives a heading a stable identity: a rename changes one row, and every reference filed under it follows. Heading ids are allocated in memory by IndexTreeModelEngine rather than by SQLite, so a heading can exist and be referenced before its row has been written.

| Field | Purpose |
|---|---|
| **id** | The heading's identifier, assigned by the tree engine rather than by autoincrement, and pointed at by project_references.heading_id. |
| **parent_id** | The parent heading's id where one has been resolved. The index tree derives its hierarchy by splitting heading_text on '!' rather than by following this column, so it is a convenience rather than the structure itself. |
| **heading_text** | The full heading path as written in the macro, with sub-levels separated by '!'. Together with depth this is the heading's identity: the lookup that finds or creates a heading matches on both. |
| **name** | Display form of the same path. Written alongside heading_text and currently identical to it. |
| **depth** | Nesting level — 0 for a main heading, 1 for a sub-entry, and so on. |

### project_references

One row per index macro occurrence found in the source, and the largest table in the database. It is the coordinate map: the character offsets recorded here are what let the application locate and rewrite the exact macro span for an entry without re-parsing the file. Offsets are character positions into newline-normalised text, not byte positions, which is what keeps them correct in files containing accented characters.

| Field | Purpose |
|---|---|
| **id** | SQLite's own row id for the occurrence. |
| **heading_id** | The project_headings row this occurrence is filed under. Set to NULL rather than cascaded if that heading is deleted, so an orphaned reference survives to be repaired. |
| **heading_raw_text** | The heading path exactly as written in this particular macro, stored alongside the id so an entry can be displayed and rewritten without a join. |
| **uid** | Textual identity of the occurrence, in the form file:line:column. Unique. |
| **unique_id_number** | The integer identifier issued by MacroIDGenerator. This is how every other subsystem — the entry table, the undo stack, the tools, the pending-changes journal — refers to this reference. |
| **file_path** | The .tex file the macro occurs in. |
| **line_number** | 1-based line of the macro within that file. |
| **column_offset** | Position of the macro on that line. |
| **absolute_position** | Character offset of the macro's opening backslash from the start of the file. This is the anchor every rewrite is calculated from, and the value that is shifted for every later macro in the file when an edit changes a span's length. |
| **absolute_end** | Character offset one past the macro's closing brace. With absolute_position it gives the exact span to replace. |
| **encap** | The page style or encapsulation for this occurrence: 'standard', the name of a command to wrap the page number in ('textbf' or 'textit' from either entry path, older aliases such as 'bold' or 'bf' from imported source), a '(' or ')' marker opening or closing a page range, or a marker with a command after it ('(textbf') for a range whose page numbers are styled. The marker always comes first, which is the form makeindex reads; grammar.split_range_encap is the only thing that separates the two halves, and every consumer asks it rather than comparing the whole value. |
| **see_references** | JSON list of see targets parsed out of the macro itself. Populated for legacy inline cross-references; managed cross-references live in their own table instead. |
| **seealso_references** | The same for see also targets. |
| **has_references** | 1 if this entry carries a real page reference, 0 if it is a cross-reference pointer with no page of its own. |
| **range_partner_id** | The unique_id_number of the other half of a page range, or NULL for an ordinary point reference. Openers and closers each point at the other. |
| **is_range_closer** | 1 if this occurrence is the ')' half of a range, 0 otherwise. |
| **macro_command** | The macro name actually used — 'index' by default, or the name of a custom indexing command the project has adopted. |

### `project_file_sync_state`

A content checksum per file, recorded only at moments when the stored coordinates are known to match that file's contents: a fresh scan, a manual resync, or a save that wrote the file itself. On the next project open each file's live checksum is compared against its recorded one, which is how an edit made while the application was closed is detected before stale coordinates can be trusted.

| Field | Purpose |
|---|---|
| `file_path` | Path of the file the checksum belongs to. Primary key. |
| `checksum` | Hash of the file's contents at the moment its coordinates were known to be correct. |
| `synced_at` | When that stamp was taken. |

### `project_custom_commands`

The custom LaTeX indexing commands this project has adopted from the global command registry. Each row is an independent snapshot taken when the command was added, so later editing the global entry does not silently change what an already-indexed project means by that command.

| Field | Purpose |
|---|---|
| `name` | The command name without its leading backslash. Primary key. |
| `body` | The command's definition as it stood when it was adopted. |
| `added_at` | When the command was added to this project. |

### `project_cross_references`

The see and see also pointers managed from the Cross-References tab. This table is authoritative for the generated cross_refs.tex: that file is rewritten in full from these rows on every add, edit and removal, and is never parsed back in. Because it cannot be reconstructed by re-scanning the source, these rows are written immediately rather than deferred to save.

| Field | Purpose |
|---|---|
| `id` | Autoincrementing row id; the cross-reference's identity in the tab and in the generated file. |
| `source_heading` | The heading the pointer is filed under — the term the reader looks up. |
| `xref_type` | 'see' or 'seealso', deciding which macro is generated. |
| `target_heading` | The heading the reader is sent to. |
| `added_at` | When the cross-reference was created. |

## 4.2 The names database

One file for the whole application, name_cache.db, written into the installed application's own data folder rather than into any project. That placement is deliberate: a personal name resolved once should stay resolved for every book the indexer works on afterwards, and a correction made on one project should not have to be made again on the next.

It backs the Name Inverter. Converting a personal name to indexed order combines two sources — a

local rule-based inversion that understands particles, generational suffixes and non-Western name orders, and a lookup against the VIAF and Library of Congress authority files over the network, with the life dates those headings carry stripped off — and the user chooses between them or overrules both. This database records the outcome. A name already present is answered from here without a network call. A lookup that found nothing is recorded as an empty value, which the reading side treats as a miss, so such a name is tried again on its next encounter rather than being written off permanently. A heading the user chose or typed is written back under the same key, so an overruled suggestion stays overruled.

It is a cache in the strict sense: nothing in it is unique, and deleting the file costs only the lookups and corrections that would have to be made again. It holds a single table.

### `cache`

One row per name that has been resolved, keyed on the name as it was originally entered. Columns added to this table after the first release are applied to an existing cache when it is opened, so a cache written by an earlier version keeps working instead of silently failing every write.

| Field | Purpose |
|---|---|
| **`key`** | The lookup key: the original, uninverted name prefixed with 'resolved:'. Primary key, so a name has exactly one cached heading. |
| **`value`** | The resolved heading in indexed order. An empty string records a lookup that produced nothing; because the reading side treats an empty value as a miss, such a name is looked up again on its next encounter and the row rewritten. |
| **`reason`** | Why the user overruled the suggestion, as one of a fixed set of codes offered by the dialog: none, patronymic, particle, regnal, mismatch or other. NULL for rows written automatically. |
| **`locale`** | The locale the conversion was made under, where one was supplied; inversion rules differ by language. |
| **`user_edited`** | 1 if the row came from a user confirmation or correction, 0 if it was written by an automatic lookup — distinguishing a heading someone vouched for from one the authority file supplied. |
| **`created_at`** | When the row was written. |